



COMPLETE DSA GUIDE: LINKEDLIST, STACK & QUEUE

From Basics to Advanced - Interview Ready Notes



TABLE OF CONTENTS

1. LinkedList - Basics to Advanced
2. Stack - Implementation & Problems
3. Queue - Types & Applications
4. Pattern Recognition Guide
5. Interview Problems (50+ Curated)
6. Complexity Cheat Sheet
7. Coding Interview Tips
8. Common Mistakes to Avoid
9. Practice Schedule
10. Final Checklist

1 LINKEDLIST - COMPLETE GUIDE

◆ WHAT IS A LINKEDLIST?

A **LinkedList** is a linear data structure where elements are stored in **nodes**, and each node points to the next node.

Array vs LinkedList:

Array: [10][20][30][40] - Contiguous memory

↓

LinkedList: [10]→[20]→[30]→[40]→null - Scattered memory

Why LinkedList?

- • Dynamic size (no fixed capacity)
- • Easy insertion/deletion at any position
- • No memory waste
- • No shifting of elements needed

When to avoid?

- • Need random access by index
- • Limited memory (extra space for pointers)
- • Frequent search operations

◆ TYPES OF LINKEDLIST

1. Singly LinkedList

[data|next] → [data|next] → [data|next] → null

Node Structure:

```
class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

```
}  
}
```

2. Doubly LinkedList

$\text{null} \leftarrow [\text{prev}|\text{data}|\text{next}] \rightleftharpoons [\text{prev}|\text{data}|\text{next}] \rightleftharpoons [\text{prev}|\text{data}|\text{next}] \rightarrow \text{null}$

Node Structure:

```
class Node {  
    int data;  
    Node prev;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.prev = null;  
        this.next = null;  
    }  
}
```

3. Circular LinkedList

$[\text{data}|\text{next}] \rightarrow [\text{data}|\text{next}] \rightarrow [\text{data}|\text{next}]$
 $\uparrow \text{_____} |$

◆ BASIC OPERATIONS - SINGLY LINKEDLIST

Complete Implementation:

```
class LinkedList {  
    Node head;  
  
    // Node class  
    class Node {  
        int data;  
        Node next;  
        Node(int data) {
```

```

        this.data = data;
        this.next = null;
    }
}

// 1. INSERT AT BEGINNING - O(1)
void insertAtBeginning(int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
}

// 2. INSERT AT END - O(n)
void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        return;
    }
    Node current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = newNode;
}

// 3. INSERT AT POSITION - O(n)
void insertAtPosition(int data, int position) {
    Node newNode = new Node(data);
    if (position == 0) {
        newNode.next = head;
        head = newNode;
        return;
    }
    Node current = head;
    for (int i = 0; i < position - 1 && current != null; i++) {
        current = current.next;
    }
    if (current == null) return;
    newNode.next = current.next;
    current.next = newNode;
}

```

```
// 4. DELETE FIRST NODE -  $O(1)$ 
```

```
void deleteFirst() {  
    if (head == null) return;  
    head = head.next;  
}
```

```
// 5. DELETE LAST NODE -  $O(n)$ 
```

```
void deleteLast() {  
    if (head == null) return;  
    if (head.next == null) {  
        head = null;  
        return;  
    }  
    Node current = head;  
    while (current.next.next != null) {  
        current = current.next;  
    }  
    current.next = null;  
}
```

```
// 6. DELETE BY VALUE -  $O(n)$ 
```

```
void deleteByValue(int key) {  
    if (head == null) return;  
    if (head.data == key) {  
        head = head.next;  
        return;  
    }  
    Node current = head;  
    while (current.next != null && current.next.data != key) {  
        current = current.next;  
    }  
    if (current.next != null) {  
        current.next = current.next.next;  
    }  
}
```

```
// 7. SEARCH -  $O(n)$ 
```

```
boolean search(int key) {  
    Node current = head;  
    while (current != null) {  
        if (current.data == key) return true;  
        current = current.next;  
    }  
}
```

```

        return false;
    }

    // 8. GET LENGTH - O(n)
    int getLength() {
        int count = 0;
        Node current = head;
        while (current != null) {
            count++;
            current = current.next;
        }
        return count;
    }

    // 9. PRINT LIST - O(n)
    void printList() {
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " → ");
            current = current.next;
        }
        System.out.println("null");
    }
}

```

◆ ESSENTIAL TECHNIQUES & PATTERNS

★ Pattern 1: TWO POINTER TECHNIQUE

Used for: **Finding middle, cycle detection, nth node from end**

```

// Find Middle Element
Node findMiddle(Node head) {
    if (head == null) return null;
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
}

```

```

    }
    return slow;
}

// Find Nth Node from End
Node nthFromEnd(Node head, int n) {
    Node first = head;
    Node second = head;
    for (int i = 0; i < n; i++) {
        if (first == null) return null;
        first = first.next;
    }
    while (first != null) {
        first = first.next;
        second = second.next;
    }
    return second;
}

```

? INTERVIEW Q: Why does slow-fast pointer find the middle?

Answer: When fast pointer reaches the end, it has traveled 2x the distance. Since slow moves at half the speed, it's exactly at the middle.

★ Pattern 2: CYCLE DETECTION (Floyd's Algorithm)

```

// Detect Cycle
boolean hasCycle(Node head) {
    if (head == null) return false;
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

```

```
// Find Start of Cycle
Node detectCycleStart(Node head) {
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow.next;
                fast = fast.next;
            }
            return slow;
        }
    }
    return null;
}
```

★ Pattern 3: REVERSAL

```
// Reverse LinkedList - Iterative - O(n)
Node reverse(Node head) {
    Node prev = null;
    Node current = head;
    Node next = null;
    while (current != null) {
        next = current.next;
        current.next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

// Reverse LinkedList - Recursive
Node reverseRecursive(Node head) {
    if (head == null || head.next == null) {
        return head;
    }
    Node newHead = reverseRecursive(head.next);
    head.next.next = head;
}
```



```

    head.next = null;
    return newHead;
}

// Reverse in Groups of K
Node reverseKGroup(Node head, int k) {
    Node current = head;
    Node prev = null;
    Node next = null;
    int count = 0;
    while (current != null && count < k) {
        next = current.next;
        current.next = prev;
        prev = current;
        current = next;
        count++;
    }
    if (next != null) {
        head.next = reverseKGroup(next, k);
    }
    return prev;
}

```

★ Pattern 4: MERGE TWO SORTED LISTS

```

Node mergeTwoLists(Node l1, Node l2) {
    Node dummy = new Node(0);
    Node current = dummy;
    while (l1 != null && l2 != null) {
        if (l1.data <= l2.data) {
            current.next = l1;
            l1 = l1.next;
        } else {
            current.next = l2;
            l2 = l2.next;
        }
        current = current.next;
    }
    current.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}

```

★ Pattern 5: REMOVE DUPLICATES

```
// Remove Duplicates from Sorted List
Node removeDuplicates(Node head) {
    Node current = head;
    while (current != null && current.next != null) {
        if (current.data == current.next.data) {
            current.next = current.next.next;
        } else {
            current = current.next;
        }
    }
    return head;
}

// Remove Duplicates from Unsorted List - O(n) with HashSet
Node removeDuplicatesUnsorted(Node head) {
    if (head == null) return null;
    HashSet seen = new HashSet<>();
    Node current = head;
    seen.add(current.data);
    while (current.next != null) {
        if (seen.contains(current.next.data)) {
            current.next = current.next.next;
        } else {
            seen.add(current.next.data);
            current = current.next;
        }
    }
    return head;
}
```

◆ TOP 20 LINKEDLIST INTERVIEW PROBLEMS

Easy Level

1. **Reverse a LinkedList** - Already covered above
2. **Find Middle of LinkedList** - Use slow-fast pointer
3. **Detect Cycle** - Floyd's cycle detection
4. **Merge Two Sorted Lists** - Already covered above

5. Remove Nth Node from End

```
Node removeNthFromEnd(Node head, int n) {
    Node dummy = new Node(0);
    dummy.next = head;
    Node first = dummy;
    Node second = dummy;
    for (int i = 0; i <= n; i++) {
        first = first.next;
    }
    while (first != null) {
        first = first.next;
        second = second.next;
    }
    second.next = second.next.next;
    return dummy.next;
}
```

6. Check if Palindrome

```
boolean isPalindrome(Node head) {
    if (head == null || head.next == null) return true;
    Node slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    Node secondHalf = reverse(slow.next);
    Node firstHalf = head;
    while (secondHalf != null) {
        if (firstHalf.data != secondHalf.data) return false;
        firstHalf = firstHalf.next;
        secondHalf = secondHalf.next;
    }
    return true;
}
```

7. Intersection Point of Two Lists

```
Node getIntersection(Node head1, Node head2) {
    if (head1 == null || head2 == null) return null;
```

```

Node a = head1, b = head2;
while (a != b) {
    a = (a == null) ? head2 : a.next;
    b = (b == null) ? head1 : b.next;
}
return a;
}

```

Medium Level

8. Add Two Numbers

```

Node addTwoNumbers(Node l1, Node l2) {
    Node dummy = new Node(0);
    Node current = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int sum = carry;
        if (l1 != null) { sum += l1.data; l1 = l1.next; }
        if (l2 != null) { sum += l2.data; l2 = l2.next; }
        carry = sum / 10;
        current.next = new Node(sum % 10);
        current = current.next;
    }
    return dummy.next;
}

```

9. Reorder List ($L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$)

```

void reorderList(Node head) {
    if (head == null || head.next == null) return;
    Node slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    Node second = reverse(slow.next);
    slow.next = null;
    Node first = head;
    while (second != null) {
        Node temp1 = first.next;

```

```

        Node temp2 = second.next;
        first.next = second;
        second.next = temp1;
        first = temp1;
        second = temp2;
    }
}

```

10. Flatten a Multilevel Doubly LinkedList

```

Node flatten(Node head) {
    if (head == null) return null;
    Node current = head;
    while (current != null) {
        if (current.child != null) {
            Node next = current.next;
            Node child = flatten(current.child);
            current.next = child;
            child.prev = current;
            current.child = null;
            while (current.next != null) {
                current = current.next;
            }
            if (next != null) {
                current.next = next;
                next.prev = current;
            }
        }
        current = current.next;
    }
    return head;
}

```

2 STACK - COMPLETE GUIDE

◆ WHAT IS A STACK?

LIFO: Last In First Out

```
Push → | 4 | ← Pop
      | 3 |
      | 2 |
      | 1 |
      -----
```

Real-life examples:

- Stack of plates
- Undo/Redo in editors
- Browser back button
- Function call stack

◆ STACK IMPLEMENTATION - Using Array

```
class Stack {
    private int[] arr;
    private int top;
    private int capacity;

    Stack(int size) {
        arr = new int[size];
        capacity = size;
        top = -1;
    }

    void push(int x) {
        if (isFull()) {
            System.out.println("Stack Overflow");
            return;
        }
        arr[++top] = x;
    }

    int pop() {
```

```

        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
        return arr[top--];
    }

    int peek() {
        if (isEmpty()) {
            System.out.println("Stack is empty");
            return -1;
        }
        return arr[top];
    }

    boolean isEmpty() { return top == -1; }
    boolean isFull() { return top == capacity - 1; }
    int size() { return top + 1; }
}

```

◆ STACK IMPLEMENTATION - Using LinkedList

```

class StackLL {
    private Node top;

    class Node { int data; Node next; Node(int d) { data = d; } }

    void push(int x) { Node n = new Node(x); n.next = top; top = n; }
    int pop() { if (top == null) return -1; int v = top.data; top = top.next; return v; }
    int peek() { return top == null ? -1 : top.data; }
    boolean isEmpty() { return top == null; }
}

```

◆ Using Java Collections

```

// Best way - Use ArrayDeque
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);

```

```
System.out.println(stack.pop()); // 20
System.out.println(stack.peek()); // 10
```

◆ STACK PATTERNS

★ Pattern 1: BALANCED PARENTHESES

```
boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') stack.push(c);
        else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((c == ')' && top != '(') || (c == '}' && top != '{') || (c == ']' && top
!= '[')) return false;
        }
    }
    return stack.isEmpty();
}
```

★ Pattern 2: NEXT GREATER ELEMENT

```
int[] nextGreater(int[] arr) {
    int n = arr.length; int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for (int i = n-1; i >= 0; i--) {
        while (!st.isEmpty() && st.peek() <= arr[i]) st.pop();
        res[i] = st.isEmpty() ? -1 : st.peek();
        st.push(arr[i]);
    }
    return res;
}
// [4, 5, 2, 25] → [5, 25, 25, -1]
```

★ Pattern 3: LARGEST RECTANGLE IN HISTOGRAM

```
int largestRectangle(int[] heights) {
    Stack<Integer> stack = new Stack<>();
```



```

int maxArea = 0, n = heights.length;
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && heights[i] < heights[stack.peek()]) {
        int h = heights[stack.pop()];
        int w = stack.isEmpty() ? i : i - stack.peek() - 1;
        maxArea = Math.max(maxArea, h * w);
    }
    stack.push(i);
}
while (!stack.isEmpty()) {
    int h = heights[stack.pop()];
    int w = stack.isEmpty() ? n : n - stack.peek() - 1;
    maxArea = Math.max(maxArea, h * w);
}
return maxArea;
}

```

★ Pattern 4: MIN STACK

```

class MinStack {
    Stack<Integer> stack = new Stack<>();
    Stack<Integer> minStack = new Stack<>();

    void push(int x) {
        stack.push(x);
        if (minStack.isEmpty() || x <= minStack.peek()) minStack.push(x);
    }
    void pop() {
        if (stack.peek().equals(minStack.peek())) minStack.pop();
        stack.pop();
    }
    int top() { return stack.peek(); }
    int getMin() { return minStack.peek(); }
}

```

◆ TOP 15 STACK PROBLEMS

1. Implement Stack using Queues

```

class StackUsingQueue {
    Queue<Integer> q = new LinkedList<>();
    void push(int x) { q.add(x); for (int i = 0; i < q.size()-1; i++) q.add(q.remove()); }
    int pop() { return q.remove(); }
    int top() { return q.peek(); }
}

```

2. Evaluate Postfix

```

int evaluatePostfix(String exp) {
    Stack<Integer> stack = new Stack<>();
    for (char c : exp.toCharArray()) {
        if (Character.isDigit(c)) stack.push(c - '0');
        else {
            int b = stack.pop(), a = stack.pop();
            switch(c) {
                case '+': stack.push(a+b); break;
                case '-': stack.push(a-b); break;
                case '*': stack.push(a*b); break;
                case '/': stack.push(a/b); break;
            }
        }
    }
    return stack.pop();
}

```

3. Infix to Postfix

```

int precedence(char c) { if (c=='^') return 3; if (c=='*' || c=='/') return 2; if (c=='+' || c=='-') return 1; return -1; }
String infixToPostfix(String exp) {
    StringBuilder res = new StringBuilder();
    Stack<Character> st = new Stack<>();
    for (char c : exp.toCharArray()) {
        if (Character.isLetterOrDigit(c)) res.append(c);
        else if (c == '(') st.push(c);
        else if (c == ')') { while (!st.isEmpty() && st.peek()!='(')
res.append(st.pop()); st.pop(); }
        else { while (!st.isEmpty() && precedence(c) <= precedence(st.peek()))

```

```

res.append(st.pop()); st.push(c); }
    }
    while (!st.isEmpty()) res.append(st.pop());
    return res.toString();
}

```

4. Stock Span

```

int[] calculateSpan(int[] prices) {
    int n = prices.length; int[] span = new int[n];
    Stack<Integer> st = new Stack<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && prices[st.peek()] <= prices[i]) st.pop();
        span[i] = st.isEmpty() ? (i+1) : (i-st.peek());
        st.push(i);
    }
    return span;
}

```

3 QUEUE - COMPLETE GUIDE

◆ WHAT IS A QUEUE?

FIFO: First In First Out

Rear → [4][3][2][1] → Front
Enqueue Dequeue

Real-life examples:

- Line at ticket counter
- Print job queue
- CPU task scheduling

- • BFS in graphs

◆ CIRCULAR QUEUE IMPLEMENTATION

```
class CircularQueue {
    int[] arr; int front, rear, size, capacity;
    CircularQueue(int k) { capacity = k; arr = new int[k]; front = rear = -1; size = 0; }
    boolean enqueue(int v) {
        if (size == capacity) return false;
        if (size == 0) front = 0;
        rear = (rear + 1) % capacity; arr[rear] = v; size++; return true;
    }
    int dequeue() {
        if (size == 0) return -1;
        int v = arr[front];
        if (front == rear) { front = rear = -1; } else { front = (front + 1) % capacity; }
        size--; return v;
    }
    int front() { return size == 0 ? -1 : arr[front]; }
    int rear() { return size == 0 ? -1 : arr[rear]; }
    boolean isEmpty() { return size == 0; }
    boolean isFull() { return size == capacity; }
}
```

◆ QUEUE USING LINKEDLIST

```
class QueueLL {
    Node front, rear;
    class Node { int data; Node next; Node(int d) { data = d; } }
    void enqueue(int x) {
        Node n = new Node(x);
        if (rear == null) { front = rear = n; return; }
        rear.next = n; rear = n;
    }
    int dequeue() {
        if (front == null) return -1;
        int v = front.data; front = front.next;
        if (front == null) rear = null;
    }
}
```

```
        return v;
    }
}
```

◆ TYPES OF QUEUES

Priority Queue

```
PriorityQueue<Integer> pq = new PriorityQueue<>(); // Min-Heap
pq.offer(30); pq.offer(10); pq.offer(20);
System.out.println(pq.poll()); // 10
```

```
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder()); // Max-Heap
```

Deque

```
Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(10); deque.addLast(20);
deque.removeFirst(); deque.removeLast();
```

◆ QUEUE PATTERNS

★ Sliding Window Maximum

```
int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length; int[] res = new int[n-k+1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (!dq.isEmpty() && dq.peekFirst() < i-k+1) dq.pollFirst();
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
        dq.offerLast(i);
        if (i >= k-1) res[i-k+1] = nums[dq.peekFirst()];
    }
    return res;
}
```

★ Queue Using Stacks

```

class QueueUsingStacks {
    Stack<Integer> s1 = new Stack<>(), s2 = new Stack<>();
    void enqueue(int x) { s1.push(x); }
    int dequeue() {
        if (s2.isEmpty()) while (!s1.isEmpty()) s2.push(s1.pop());
        return s2.isEmpty() ? -1 : s2.pop();
    }
}

```

★ First Non-Repeating in Stream

```

void firstNonRepeating(String stream) {
    Queue<Character> q = new LinkedList<>(); int[] freq = new int[26];
    for (char c : stream.toCharArray()) {
        q.offer(c); freq[c-'a']++;
        while (!q.isEmpty() && freq[q.peek()-'a'] > 1) q.poll();
        System.out.print(q.isEmpty() ? "-1 " : q.peek()+" ");
    }
}

```

★ Level Order Traversal (BFS)

```

void levelOrder(TreeNode root) {
    if (root == null) return;
    Queue<TreeNode> q = new LinkedList<>(); q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            System.out.print(node.val + " ");
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        System.out.println();
    }
}

```

◆ TOP 15 QUEUE PROBLEMS

1. Generate Binary Numbers

```
void generateBinary(int n) {
    Queue<String> q = new LinkedList<>(); q.offer("1");
    for (int i = 0; i < n; i++) {
        String curr = q.poll(); System.out.println(curr);
        q.offer(curr+"0"); q.offer(curr+"1");
    }
}
```

2. Rotten Oranges

```
int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> q = new LinkedList<>(); int fresh = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) {
            if (grid[i][j] == 2) q.offer(new int[]{i,j});
            else if (grid[i][j] == 1) fresh++;
        }
    if (fresh == 0) return 0;
    int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}}; int mins = 0;
    while (!q.isEmpty()) {
        int size = q.size(); mins++;
        for (int i = 0; i < size; i++) {
            int[] curr = q.poll();
            for (int[] d : dirs) {
                int x = curr[0]+d[0], y = curr[1]+d[1];
                if (x>=0 && x<cols && y>=0 && y<rows && grid[x][y] == 1) {
                    q.offer(new int[]{x,y});
                    grid[x][y] = 2;
                }
            }
        }
    }
    return mins;
}
```

3. Gas Station

```
int canCompleteCircuit(int[] gas, int[] cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        total += gas[i] - cost[i];
        tank += gas[i] - cost[i];
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }
    return total >= 0 ? start : -1;
}
```

```

        if (tank < 0) { start = i+1; tank = 0; }
    }
    return total >= 0 ? start : -1;
}

```

4. LRU Cache

```

class LRUCache {
    Map<Integer, Node> cache = new HashMap<>();
    int capacity; Node head, tail;
    class Node { int key, value; Node prev, next; Node(int k, int v) { key=k;
value=v; } }
    LRUCache(int cap) {
        capacity = cap;
        head = new Node(0,0); tail = new Node(0,0);
        head.next = tail; tail.prev = head;
    }
    int get(int key) {
        if (!cache.containsKey(key)) return -1;
        Node n = cache.get(key); remove(n); insertHead(n);
        return n.value;
    }
    void put(int key, int val) {
        if (cache.containsKey(key)) remove(cache.get(key));
        if (cache.size() == capacity) { cache.remove(tail.prev.key);
remove(tail.prev); }
        Node n = new Node(key, val); insertHead(n); cache.put(key, n);
    }
    void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    void insertHead(Node n) { n.next = head.next; n.prev = head; head.next.prev =
n; head.next = n; }
}

```


4 PATTERN RECOGNITION GUIDE

HOW TO IDENTIFY WHICH DATA STRUCTURE?

Data Structure	Use When	Keywords
LinkedList	Frequent insert/delete, No random access	reverse, cycle, middle, merge, nth from end
Stack	LIFO, Matching pairs, Backtracking	balanced, next greater, histogram, undo, parentheses
Queue	FIFO, BFS, Sliding window	level order, rotting, first non-repeating, shortest path
Deque	Operations at both ends	sliding window max/min
Priority Queue	Need min/max repeatedly	kth largest, merge k sorted, scheduling

Use **LINKEDLIST** when: ☒ Frequent insertions/deletions ☒ Don't need random access ☒

Implement Stack/Queue ☒ Two pointer problems ☒ Reverse operations

Use **STACK** when: ☒ Need LIFO behavior ☒ Matching pairs ☒ Next greater/smaller ☒

Expression evaluation ☒ Backtracking ☒ Undo operations

Use **QUEUE** when: ☒ Need FIFO behavior ☒ Level order traversal ☒ Sliding window ☒ First come first serve ☒ Stream processing

5 COMPLETE INTERVIEW PROBLEM SET

🔥 LINKEDLIST - 25 Must-Do

Easy (10):

1. ☒ Reverse LinkedList
2. ☒ Find Middle Element
3. ☒ Detect Cycle
4. ☒ Remove Duplicates from Sorted
5. ☒ Merge Two Sorted Lists

6. ☒ Check if Palindrome

7. ☒ Intersection Point

8. ☒ Remove Nth from End

9. Delete a Node (given only node)

10. Convert Binary in LinkedList to Integer

Medium (10):

11. ☒ Add Two Numbers

12. ☒ Reorder List

13. ☒ Remove Duplicates from Unsorted

14. ☒ Reverse in K Groups

15. Partition List

16. Odd Even LinkedList

17. Sort List (Merge Sort)

18. Rotate List

19. Swap Nodes in Pairs

20. Copy List with Random Pointer

Hard (5):

21.  Flatten Multilevel Doubly LinkedList

22. Merge K Sorted Lists

23. Reverse Nodes in k-Group

24. LRU Cache

25. Design Browser History

STACK - 20 Must-Do

Easy (8):

1. ☒ Valid Parentheses
2. ☒ Implement Stack using Array
3. ☒ Implement Stack using LinkedList
4. ☒ Implement Stack using Queues
5. ☒ Min Stack

6. Remove All Adjacent Duplicates

7. Backspace String Compare

8. Build Array with Stack Operations

Medium (8):

9. ☒ Next Greater Element I & II

10. ☒ Largest Rectangle in Histogram

11. ☒ Evaluate Postfix Expression

12. ☒ Infix to Postfix

13. ☒ Stock Span Problem

14. Daily Temperatures

15. Simplify Path

16. Decode String

Hard (4):

17. Trapping Rain Water

18. Maximum Rectangle in Binary Matrix

19. Maximal Rectangle

20. Basic Calculator I, II, III

 **QUEUE - 20 Must-Do**

Easy (7):

1. ☒ Implement Queue using Array
2. ☒ Implement Queue using LinkedList
3. ☒ Implement Queue using Stacks
4. ☒ Circular Queue
5. ☒ Generate Binary Numbers
6. Number of Recent Calls
7. Design Circular Deque

Medium (9):

8. ☒ Sliding Window Maximum

9. ☒ First Non-Repeating in Stream

10. ☒ Rotten Oranges

11. ☒ Gas Station (Circular Tour)

12. Task Scheduler

13. Jump Game VI

14. Binary Tree Level Order

15. Binary Tree Zigzag Level Order

16. Shortest Path in Binary Matrix

Hard (4):

17. ☒ LRU Cache

18. LFU Cache

19. Sliding Window Median

20. Design Snake Game

6 COMPLEXITY CHEAT SHEET

LinkedList Operations

Operation	Singly	Doubly	Time
Access by index	✗	✗	$O(n)$
Insert at beginning	✓	✓	$O(1)$
Insert at end	✓	✓	$O(n)$ / $O(1)$ with tail
Delete at beginning	✓	✓	$O(1)$

Delete at end	✗	✓	$O(n)$ / $O(1)$ with tail
Search	✗	✗	$O(n)$
Reverse	✓	✓	$O(n)$

Stack Operations

Operation	Array	LinkedList	Time
Push	✓	✓	$O(1)$
Pop	✓	✓	$O(1)$
Peek	✓	✓	$O(1)$
isEmpty	✓	✓	$O(1)$
Size	✓	✓	$O(1)$

Queue Operations

Operation	Array	LinkedList	Circular	Time
Enqueue	✗	✓	✓	$O(1)$
Dequeue	✗	✓	✓	$O(1)$

Front	✓	✓	✓	0(1)
isEmpty	✓	✓	✓	0(1)
Size	✓	✓	✓	0(1)

7 CODING INTERVIEW TIPS

Before Writing Code

1. Clarify the problem

- Ask about edge cases
- Confirm input/output format
- Check constraints (size, values)

2. Think out loud

- Explain your approach • Discuss time/space complexity • Mention alternatives

3. Choose right data structure

- Use pattern recognition • Consider trade-offs

While Writing Code

1. **Start with brute force** - Get working solution first, Then optimize

2. **Use meaningful names** - current, prev, next for LinkedList; top for Stack; front, rear

for
Queue

3. **Handle edge cases** - Empty list/stack/queue, Single element, null pointers

After Writing Code

1. **Test with examples** - Normal case, Edge cases, Large input
2. **Analyze complexity** - Time: $O(?)$, Space: $O(?)$
3. **Optimize if needed** - Can we do better? Trade time for space or vice versa

8 COMMON MISTAKES TO AVOID

✗ Not checking for null

✗ Losing reference to head

✗ Infinite loop in cycle

✗ Not handling single node case

✗ Forgetting to update next pointers

✓ Always check null before accessing `.next`

✓ Use dummy node to simplify edge cases

✓ Draw the pointers on paper

Stack Mistakes

✗ Not checking if stack is empty before `pop()`

✗ Using `Stack` class instead of `Deque`

✗ Not handling overflow/underflow

✗ Wrong order in expression evaluation

✓ Always check `isEmpty()` before operations

✓ Use `ArrayDeque` for `Stack` in production

Queue Mistakes

✗ Not checking if queue is empty before `dequeue()`

✗ Confusing front and rear

✗ Circular queue wrap-around logic errors

✗ Not handling full queue case

✓ Always check `isEmpty()` before operations

✓ Use `% capacity` for circular queue

Week 1-2: LinkedList Basics

- Day 1-2: Implementation (Insert, Delete, Search)
- Day 3-4: Reversal patterns
- Day 5-6: Two pointer technique
- Day 7: Cycle detection
- Day 8-10: Easy problems (10 problems)
- Day 11-14: Medium problems (10 problems)

Week 3: Stack Mastery

- Day 1-2: Implementation
- Day 3-4: Parentheses & Expression problems
- Day 5-6: Next Greater Element pattern
- Day 7: Histogram problems
- Day 8-10: Easy problems (8 problems)
- Day 11-14: Medium problems (8 problems)

Week 4: Queue & Advanced

- Day 1-2: Queue implementation
- Day 3-4: BFS & Level order
- Day 5-6: Sliding window with Queue
- Day 7: Priority Queue problems
- Day 8-10: Easy problems (7 problems)
- Day 11-14: Medium problems (9 problems)

Week 5-6: Mixed Problems

- Solve 3-5 problems daily
- Mix of all three data structures

- Focus on hard problems
- Mock interviews



FINAL CHECKLIST

Before your interview, make sure you can:

LinkedList:

- [] Implement from scratch
- [] Reverse iteratively & recursively

- [] Find middle using slow-fast pointers

- [] Detect and remove cycle

- [] Merge two sorted lists

- [] Check if palindrome

Stack:

- [] Implement using array & LinkedList

- [] Solve valid parentheses

- [] Find next greater element

- [] Evaluate postfix expression

- [] Implement min/max stack

Queue:

- [] Implement using array & LinkedList
- [] Implement circular queue
- [] Solve sliding window maximum
- [] Perform BFS traversal
- [] Implement queue using stacks

General:

- [] Identify pattern from problem statement
- [] Write clean, bug-free code in 15-20 mins
- [] Analyze time & space complexity

- [] Handle all edge cases
- [] Explain approach clearly



RESOURCES

Practice Platforms:

- LeetCode: LinkedList (Easy → Medium → Hard)
- HackerRank: Stacks and Queues section
- GeeksforGeeks: Data Structures track
- InterviewBit: Programming section

YouTube Channels:

- Abdul Bari (Theory)
- TakeUForward (Striver)
- NeetCode (Problem solving)
- mycodeschool (Classic explanations)

Books:

- Cracking the Coding Interview
- Elements of Programming Interviews



REMEMBER

**"Practice doesn't make perfect. Perfect
practice
makes perfect."**

- Solve 5-10 problems daily
- Understand the pattern, don't memorize
- Revise solved problems weekly
- Time yourself (20 mins per problem)
- Focus on writing clean code

You've got this! Happy Coding! 💪

